



ARDUINO PHYSICAL AI CHALLENGE INDIA 2026

Project Report

Organised by Robu.in × Arduino

Project Title	VAHA — Offline-First Physical AI Voice Capture Device
Team Name	VAHA
Registration / Team ID	APC-2026-AP-13507
Contest Track	Smart Homes / Consumer AI
Institution & City	Dharanova Pvt Ltd, Visakhapatnam, Andhra Pradesh

Team Members (1-4)

Name	Email	Role
Ritesh Bonthalakoti	ritesh@dharanova.com	Solo Developer
Member 2 (optional)		
Member 3 (optional)		
Member 4 (optional)		

1. Project Overview

Problem Statement

Creative minds don't switch off. Ideas show up at the office, school, the park, a beach, in transit — anywhere you'd normally have a phone or notebook close by. There's one place that isn't true: the bathroom, where the mind keeps wandering with no way to write anything down. VAHA closes that gap — built for the one place your other devices can't follow you, so an idea never has to wait.

How Your Project Works

- Say "Marvin" and VAHA starts listening — no phone, no button, no unlocking anything.
- Say "I'm done" and it stops, transcribes what you said, and saves it — entirely on the device, no cloud, no internet.
- While it listens, VAHA also tracks water consumption and TVOC (air quality) in the background, so every idea comes with a snapshot of the room it happened in.
- Data chunking, on-device AI, speech-to-text, text-to-speech, and sensor processing all run on the Arduino UNO Q itself.
- VAHA syncs privately with the companion app over your own Wi-Fi — your voice and data are never sent to the cloud or the internet.

Privacy & Safety — VAHA is not always recording.

- **Nothing is captured until the wake word.** VAHA continuously listens only for the single trained keyword "Marvin." Until that word is detected, audio is processed in a rolling buffer for keyword-spotting alone and is never written to disk, never transcribed, and never leaves the board.
- **Recording is explicit, bounded, and visible.** A capture begins only after the wake word fires, and ends on "I'm done" or a Voice Activity Detection (VAD) silence timeout. A chime confirms start and stop, so the user always knows when VAHA is recording.
- **Nothing ever reaches the cloud.** Wake-word detection, transcription, and storage all run on the Arduino UNO Q itself. Sync happens over the user's own Wi-Fi to their own phone — there is no external server, no account, and no third-party service in the path.
- **Built for the most private room in the house.** Because VAHA is designed for the bathroom, this guarantee is the product, not a feature. A device that could not prove it was silent by default would have no business being there.

Why These Sensors? — The Bathroom Is a Blind Spot

The bathroom is the room we use most and measure least — no screen, no smart display, usually nothing keeping track at all. VAHA is already in there listening for your ideas, so it only made sense to have it read the environmental parameters of the room at the same time.

- **Water goes untracked where it's used most.** A monthly bill is one number for the whole house. The Hall-effect flow sensor gives a real per-event figure — this shower, this tap, this leak.
 - **Damp bathrooms become unhealthy bathrooms.** Sustained humidity after every shower is what drives mould and material damage. The DHT22 Sensor shows whether the room actually dries out between uses.
 - **Air quality in a closed room is invisible.** Cleaning products and aerosols concentrate in a small unventilated space. Instead of guessing by smell, the AGS02MA TVOC sensor makes that load visible.
 - **Context makes the notes worth more.** Every capture is stamped with water, temperature, humidity, and the TVOC reading (Total Volatile Organic Compounds) from that moment — turning voice notes into a record of the room itself.
-

Why Arduino UNO Q?

- Combines a Linux-capable processor with a microcontroller on one board, linked through the Arduino Bridge API.
- The Linux side runs the full on-device AI pipeline: wake-word detection, stop-phrase detection, Voice Activity Detection (VAD) for silence-based cutoff, and speech-to-text.
- The microcontroller side handles the physical sensors (DHT22, water flow, TVOC) and exposes them via `Bridge.provide("sensors_get", ...)`.
- One board captures both your voice and a Room Reading of what's around it — no cloud service needed for any of it.

2. Components Used (BOM)

Component	Qty
Arduino UNO Q (ABX00087) — 4GB RAM / 32GB storage	1
DHT22 Temperature & Humidity Sensor	1
Water flow sensor — Hall-effect pulse output (7.5 pulses/L/min)	1
AGS02MA TVOC / air quality sensor	1
Collar mic (USB, via CS202 audio adapter)	1
USB audio output device (chime / TTS playback via Piper)	1
Jumper wires / wiring harness	~
Enclosure / mounting hardware	1
Portronics USB-C hub (multiport adapter — mic/audio + peripheral connections)	1
Mini breadboard (sensor wiring)	2
Power bank (10000 mAh, portable power for testing/demo)	1
PAM8403 audio amplifier module	1
Speaker (4 Ω / 3 W) — chime + TTS output	1

3. System Architecture & Circuit

Step-by-Step Workflow

- 1. Wake word** — User says “Marvin.” The on-device Edge Impulse model (new-marvin.eim) detects it and a capture_started event goes out over WebSocket.
 - 2. Recording** — Raw audio is captured continuously at **48 kHz** (16-bit PCM) with light noise reduction applied.
 - 3. Stop condition** — Voice Activity Detection (VAD) ends the recording when the trained stop phrase “im_done” is detected, or after **~10 seconds** of silence.
 - 4. Transcription** — Audio is downsampled to **16 kHz** and transcribed fully on-device using faster-whisper small (**277M** params, fine-tuned for VAHA).
 - 5. Sensor snapshot** — Temperature, humidity, flow, and TVOC readings are pulled via the sketch's sensors_get() bridge call and attached to the capture.
 - 6. Local storage** — The capture (audio.wav, transcript.json, metadata.json, checksum.md5) is written to captures/YYYY/MM/DD/uuid/ on the device, optionally synced to Notion, and exposed to the mobile app through the FastAPI edge server.
 - 7. Sync to app** — The companion app pulls the capture over local Wi-Fi, verifies the checksum, stores it in SQLite, and the device clears its local copy.
-

Block Diagram & Circuit Schematic

VAHA — System Block Diagram

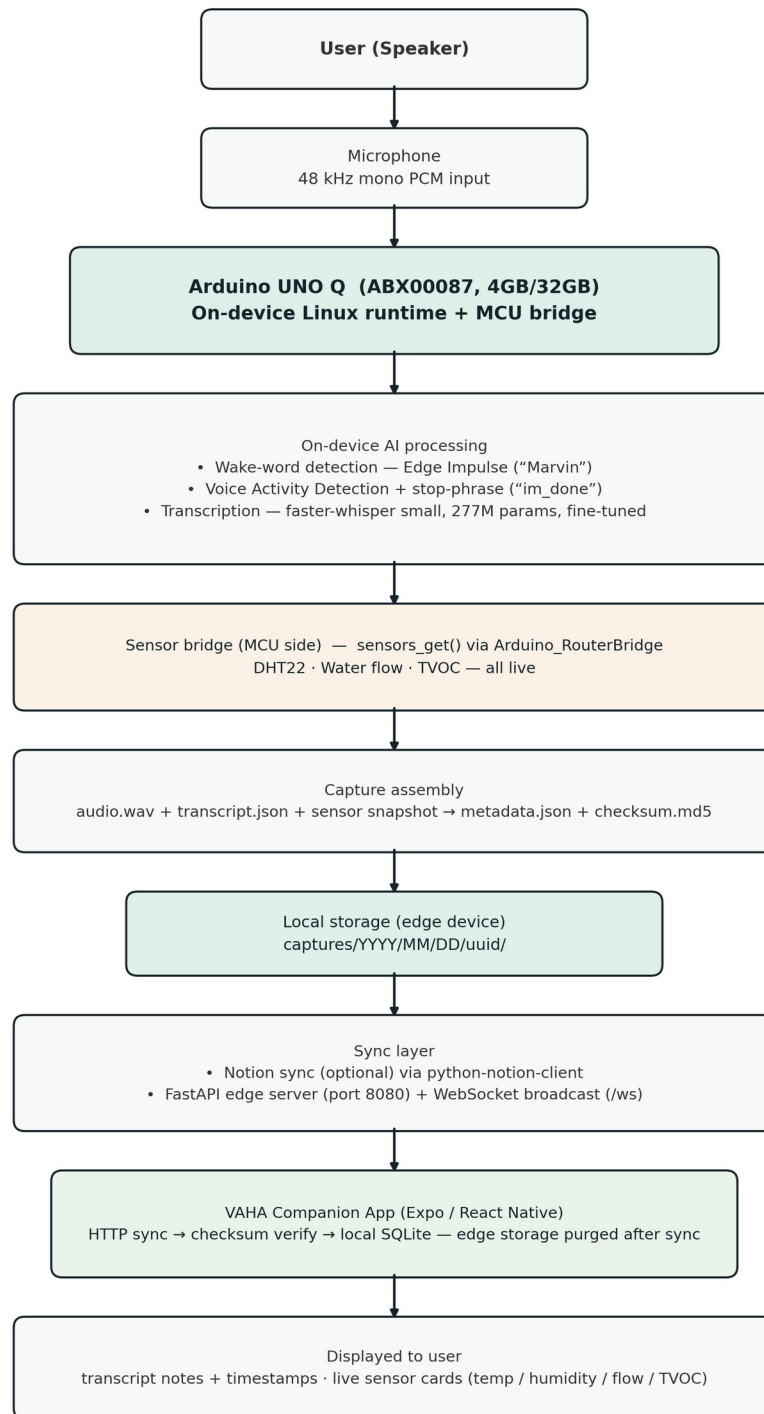


Figure 1. VAHA system block diagram — wake word, VAD, on-device transcription, sensor bridge, local storage, and sync path.

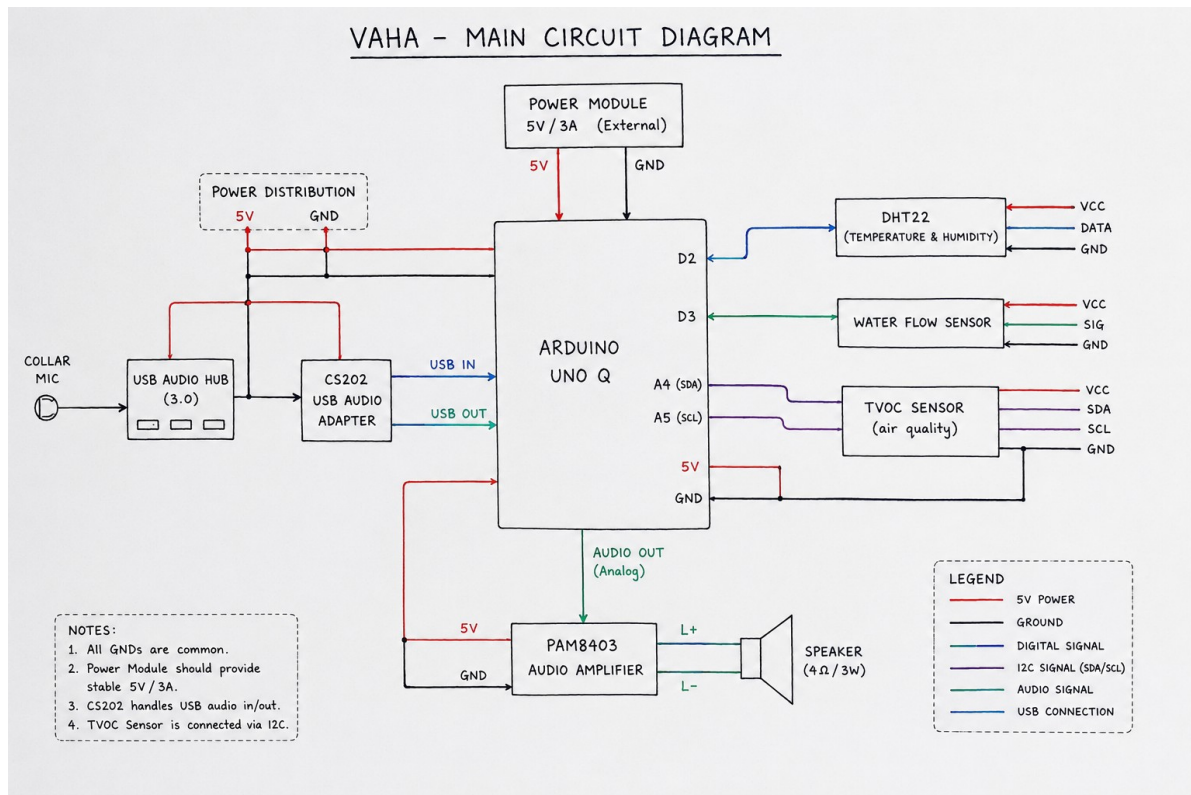


Figure 2. VAHA main circuit diagram – UNO Q pin mapping for DHT22, water-flow sensor, TVOC (I2C), USB audio hub/CS202 adapter, and speaker amplifier.

4. AI / ML Model Details

Model Used	faster-whisper small (277M params) — speech-to-text Edge Impulse keyword-spotting — wake word + stop phrase
Training Platform	Edge Impulse Studio (wake-word / stop-phrase model) Fine-tuned on-device pipeline (speech-to-text)
Accuracy	98.7% validation accuracy (Edge Impulse, wake-word / stop-phrase model)
Dataset	1,800+ audio samples (custom-recorded + augmented)

Brief Description & Limitations

- Wake word (“Marvin”) and stop phrase (“I'm done”) detected by a custom Edge Impulse keyword-spotting model — **98.7%** validation accuracy.
- Speech-to-text runs fully on-device using faster-whisper small (**277M** params), fine-tuned for VAHA.
- Voice Activity Detection (VAD) ends a recording automatically after ~**10 seconds** of silence, even without the stop phrase.
- Trained on **1,800+** audio samples (custom-recorded + augmented).
- Known limitation: accuracy drops in very high background noise (e.g., running shower/exhaust fan) — no formal noise-robustness benchmark logged yet.

5. Code Structure

sketch/sketch.ino (microcontroller side)

- setup() — initialises DHT22, flow sensor interrupt, and the AGS02MA (TVOC) I2C sensor.
- loop() — polls sensors on a timer and updates the latest readings.
- sensors_get() — exposed to the Linux side via Bridge.provide(), returns the current sensor snapshot as JSON.

python/main.py (Linux side — orchestration)

- Loads the Edge Impulse wake-word/stop-phrase model and the faster-whisper model at startup.
- find_best_mic() — auto-detects the connected USB mic (CS202 adapter).
- Runs the capture state machine: idle → listening → recording → transcribing → saved.

python/captures.py

- Writes each capture to captures/YYYY/MM/DD/uuid/ — audio.wav, transcript.json, metadata.json, checksum.md5.
- Optionally pushes the capture to Notion via python-notion-client.

FastAPI edge server

- Serves capture data over HTTP on port **8080** and broadcasts live events over WebSocket (/ws) for the mobile app.

mobile/src/ (companion app — Expo / React Native)

- Syncs captures from the device over local Wi-Fi, verifies each checksum, and stores them in a local SQLite database.
- Displays transcripts and live sensor cards (temperature, humidity, flow, TVOC).

GitHub Repository	https://github.com/dharanova-code/vaha
Demo Video Link	https://youtu.be/_Y_hMeHslhI?si=Uxo2WZzAGikBSO-f

6. Testing & Results

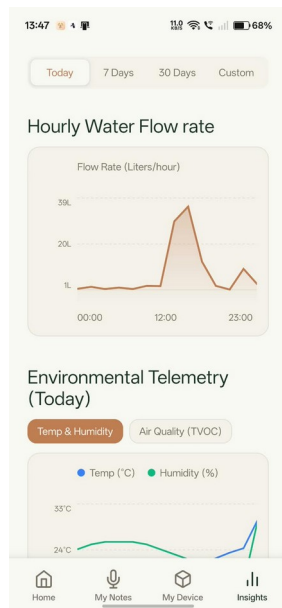
- ✓ End-to-end capture flow verified repeatedly: wake word → recording → stop phrase / VAD cutoff → transcription → local save → app sync.
- ✓ Sensor readings (DHT22 temperature/humidity, water flow, TVOC) verified live and logging correctly with every capture.
- ✓ Checksum verification confirmed on every capture synced to the companion app — no corrupted transfers observed.
- ✓ Wake-word / stop-phrase model validated at **98.7%** accuracy in Edge Impulse Studio on the held-out test set.
- ✓ Notion sync and FastAPI/WebSocket broadcast tested and working for real-time capture updates.
- ✓ **Verified under real bathroom noise — 34 capture trials.** Wake word and stop phrase were exercised across **34 capture** attempts in the actual bathroom with the exhaust fan running and the tap open, not only in quiet conditions. Detection held reliably throughout; these runs set the current **0.75 / 0.80** thresholds.
- **Known gap.** Wake-word and capture reliability are verified in real conditions, but a formal word-error-rate benchmark for the transcription stage has not yet been logged.

Companion App — Live Output

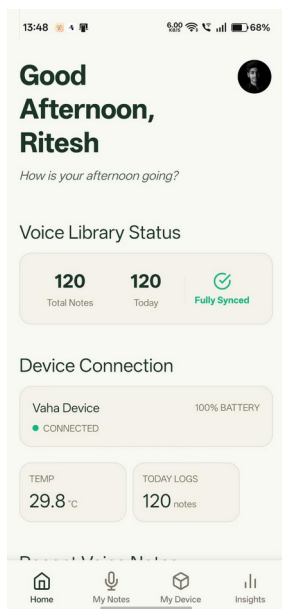
Captured from the VAHA companion app running against the live device on the local network (192.168.88.28). All readings and transcripts below are produced on-device.



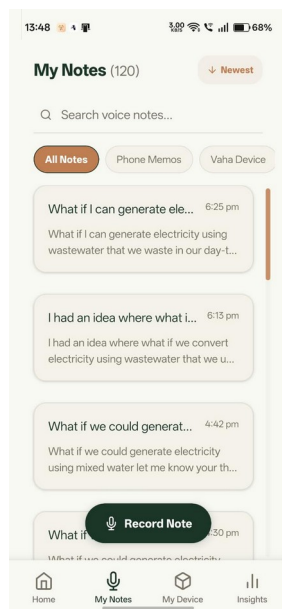
Device tab — live sensor readings straight off the UNO Q: 29.8 °C, 73 % humidity, 96 ppb TVOC, 5.0 L/m flow, 2.50 L total.



Insights tab — hourly water-flow rate and environmental telemetry trended over the day.



Home tab — 120 captures logged and fully synced from device to phone during a single test session.

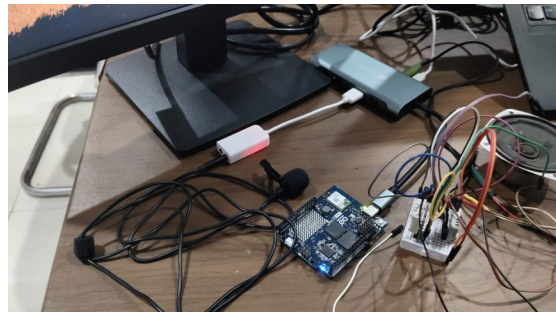


Notes tab — on-device transcripts of real captures, searchable and filterable by source.

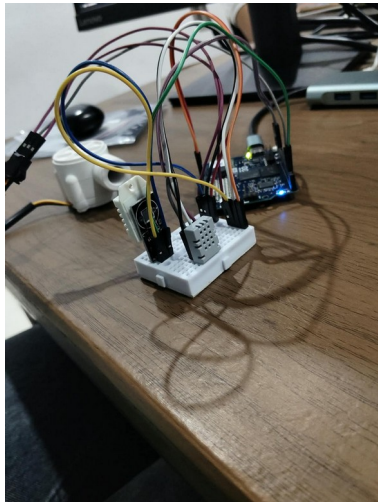
Project Images



The VAHA build workspace — UNO Q, sensors, and wiring during assembly.



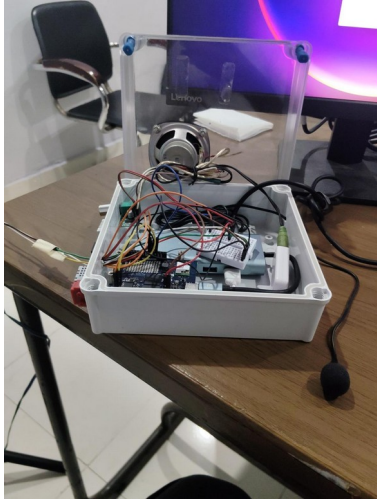
USB microphone and Arduino UNO Q wired for wake-word capture.



DHT22 and TVOC sensors on the breadboard sensor rig.



Water-flow sensor wired in-line for live flow tracking.



Early VAHA enclosure with all components mounted together.



VAHA device with the Dharanova brand nameplate, before final assembly.



Fully enclosed VAHA unit — external mic input, DHT22 vents, and flow-sensor port.

7. Challenges, Learnings & Future Improvements

Challenges Faced

- Getting the TVOC sensor (AGS02MA) to initialise reliably over I2C took real iteration — now resolved, and it reports live readings alongside DHT22 and the flow sensor.
- **Getting VAHA to hear you over the room.** A bathroom is acoustically hostile: a running tap, a shower, and an exhaust fan all produce broadband noise, and hard tile surfaces add heavy reverberation that smears speech. The wake-word and stop-phrase thresholds (**0.75** and **0.80**) had to be tuned to fire reliably through that noise without false-triggering on it — which is why the Edge Impulse dataset was recorded in the real environment rather than in a quiet room.
- Getting speech-to-text to run smoothly on-device required fine-tuning faster-whisper small rather than relying on the stock pretrained model.
- Reliably auto-detecting the correct USB microphone (CS202 adapter) across reboots and different port orders.
- Keeping the whole pipeline — wake word, VAD, transcription, sensor polling — responsive on a single board without needing the cloud.

What You Learned & What's Next

- Built VAHA: a fully offline, hands-free idea-capture device that also logs the room's water use and air quality with every note — no cloud, no app dependency to function.
- Learned that on-device AI on the UNO Q is genuinely capable of running wake-word detection, VAD, and speech-to-text together in real time, without offloading to the cloud.
- Learned that reliable hardware (sensor init, mic detection) needs as much iteration as the AI pipeline.
- Next: add a formal accuracy/latency benchmark suite for transcription and wake-word detection.
- **Next — hearing clearly through shower, fan, and tile echo.** Planned work: expand the Edge Impulse dataset with recordings captured mid-shower, tap running, and fan on; add adaptive noise-floor estimation so thresholds shift with ambient level; and evaluate a second microphone for speaker separation.
- Next: move from prototype breadboard wiring to a compact custom PCB for a production-ready enclosure.

8. Declaration

We confirm this is our original, unpublished work. The Arduino® UNO™ Q is the primary board in this project. All team members have reviewed and agree to this report.

Date	17 August 2026
-------------	----------------

Arduino Physical AI Challenge India 2026 · Robu.in × Arduino · contest@robu.in